

The Notebook You Read Is Not the Notebook That Ran: Execution-Order Reconstruction for LLM Understanding of Jupyter Notebooks

Author: MD. Raiyan Bin Rafique

Affiliation: Department of Computer Science and Engineering, Rajshahi University of Engineering and Technology, Rajshahi, Bangladesh

Corresponding author: MD. Raiyan Bin Rafique, raiyanrohit10@gmail.com

ORCID: Not available.

Abstract

Context: A Jupyter notebook stores cells in *visual* order, which often differs from the *execution* order they actually ran in. Its JSON format also buries code under base64 images, duplicate MIME formats, and metadata. Stripping content saves tokens but also loses clues needed to recover true execution order. **Objective:** We test whether reconstructing execution order improves an LLM’s ability to track program state, both in controlled and real settings. **Method:** We built JupPreprocessor, a serializer that never calls an LLM: it resolves what it can recover statically and flags what it cannot. We evaluated it on a synthetic benchmark spanning fifteen models and seven vendors, and on ground truth from re-executing 112 real crashed notebooks (JunoBench) under both orderings. **Results:** Unaided, models resolve state correctly 0%–97% of the time; our fix scores 100% at roughly 95% fewer tokens. On 52 real order-divergent variables, however, our serializer shows no per-item accuracy advantage over the raw file for gpt-4o, our one answer model on this corpus, once it answers (23–27% correct, overlapping 95% confidence intervals), a result that held after we fixed a grading bug. The defensible advantage is narrower: the raw file is too large to fit in context for 62% of these items, versus 15–17% for the compressed formats. **Conclusions:** Execution-order reconstruction is necessary and sufficient for state-tracking in a controlled setting. On real notebooks, our evidence shows only that the serializer keeps more notebooks readable, not that answers become more accurate once a file is readable.

Keywords: Jupyter notebooks; large language models; execution-order reconstruction; program state serialization; empirical software engineering; notebook reproducibility

Declarations

Funding. No external funding was received for this work.

Competing interests. The author declares no competing financial or non-financial interests directly or indirectly related to this work.

Author contributions. Single-author manuscript: MD. Raiyan Bin Rafique conceived the study, designed and ran all experiments (with the LLM-assistance disclosed in the manuscript’s Methods section), analyzed the results, and wrote the manuscript.

Ethics approval. This study did not involve human participants, animal subjects, or personally identifying data; no ethical approval was required. All notebook data is drawn from JunoBench, a publicly released, pre-existing benchmark of crashed machine-learning notebooks (Wang et al., 2025).

Consent to participate / Consent for publication. Not applicable; this study involved no human participants.

Data, materials, and/or code availability. The complete source code for JupPreprocessor, the evaluation harnesses (Notebook-NIAH, the real-notebook dual-order ground-truth extractor, the Themisto and JunoBench evaluation scripts), all measured reports underlying every quantitative claim in this manuscript, and the full project changelog are available at <https://github.com/hodini007/notebook-research>.